- Melissa 마음 정리 다이어리 -

Ⅰ. 연구배경 및 목적 ·····
(1) 연구 소개
(2) 연구 목적 및 이유

Ⅱ. 수업 내용 외 활용한 내용 소개 ·····
(1) 파일 구조
(2) 활용 기술 소개

Ⅲ. 팀원들의 역할 분담 ·····

Ⅳ. 기대효과 및 활용분야 ·····

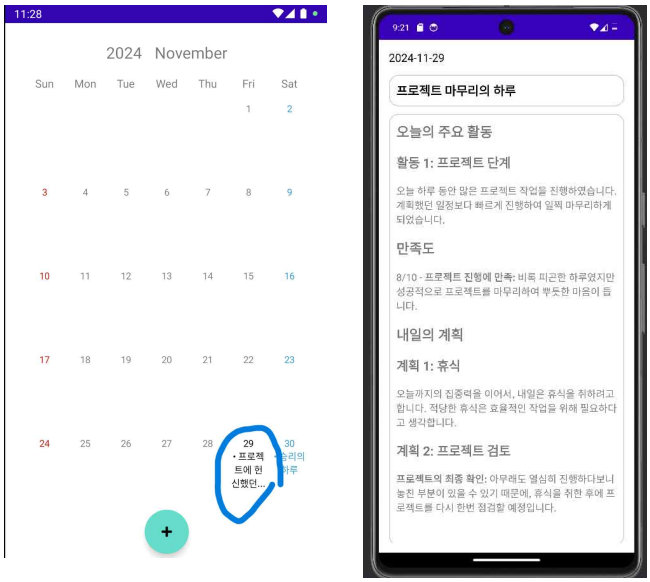
Ⅴ. 참고자료 ·····

소속(전공)	정보융합학부
과목명	모바일 프로그래밍
학번 / 이름	2023204059 강주영, 2023204078 김다솔, 2023204089 박민석, 2023204090 박준수
제출일	2024/ 12/ 02 (월)

1.연구 배경 및 목적

1-(1) 연구 소개

본 연구는 사용자가 하루 동안 느낀 감정과 경험을 정리하고, 대화를 통해 생각을 정리하도록 돕는 그리드 기반 디지털 다이어리 앱 '멜리사'를 개발하는 것을 목적으로 한다. 멜리사는 사용자와의 대화를 통한 텍스트 기록을 중심으로 정신 건강 관리와 디지털 디톡스를 지원하기 위해 설계되었으며, 사용자는 멜리사와의 대화를 통해 하루를 되돌아보고 정리할 수 있다. 앱의 주요 메커니즘은 사용자가 멜리사를 부름으로써 대화가 시작되며, 채팅방으로 화면이 넘어가면 멜리사가 먼저 질문을 던지고, 사용자가 이에 답하며 하루를 정리해 나가는 방식으로 구성된다. 예를 들어, 멜리사는 다음과 같은 질문을 통해 대화를 유도한다: "오늘 뭐 했어?", "오늘 아쉬운 점이 뭐야?", "어떻게 하면 더 괜찮아질 것 같아?" 이런 식으로 계속 사용자와 대화를 유도하고, 답변에 대해 꼬리물기식으로 질문을 계속하여 사용자가 자신의 감정과 하루를 정리하도록 돕는다. 대화를 마친 후에는, 멜리사가 자동으로 대화 내용을 요약하여서 저장해준다. 저장된 후, 메인 화면(달력)에 그날 일자에 다음 이미지와 같이 그날 하루에 대한 간략한 활동이 요약되어서 보여지고, 그 날짜를 들어가면 멜리사가 요약한 그날 하루 활동에 대한 제목, 주요 활동, 만족도, 내일의 계획이 표시되어서, 그날 있었던 대화를 되새겨볼 수 있다.



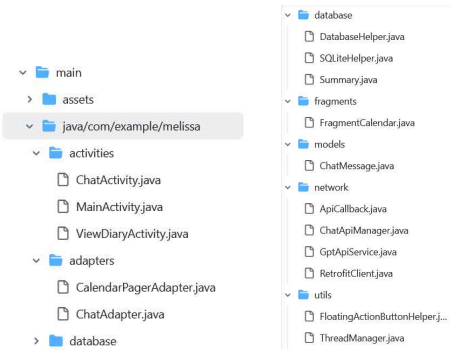
[그림 1, 2] 멜리사가 요약해준 하루 요약 제목 / 멜리사가 요약해준 하루

1-(2) 연구 목적 및 이유

본 연구는 '기록'에 초점을 두고 진행되었다. 우리는 대학생 및 20~30대 연령층이 특히 일기 쓰기의 필요성을 느끼고 있다는 사실을 글로쓰 팀의 설문조사[1]와 알바몬의 설문조사[2] 결과에서 확인할 수 있었다. 그러나, 글로쓰 팀 설문조사 결과에 따르면, '시간이 없다', '귀찮다'는 이유로 인해 실제로 일기를 꾸준히 작성하는 사람은 많지 않았다는 사실을 확인하였다. 실제로 바쁜 20~30 대의 대학생, 직장인들이 매일 꾸준히 일기를 작성하기란 쉽지 않은 일이다. 즉, 필요성과 욕구는 존재하지만 이를 실천으로 옮기기에는 현실적인 어려움이 따르고 있다는 것이다. 우리 팀은 이러한 문제를 해결하기 위해, 어떻게 하면 최소한의 노력으로 하루를 기록할 수 있을지에 대해 고민하였다. 그 결과, '대화'를 통한 기록에 초점을 맞추게 되었다. 이는 단순히 일처럼 기록하는 방식이 아니라, 친구와 대화하듯이 감정을 돌아보고, 대화 내용이 자동으로 저장되는 방식을 제안하는 것이다. 이러한 생각에 기반하여 우리는 안드로이드 어플 '멜리사'를 개발하게 되었고, 그 결과 사용자는 자기 전에 앱에 접속해 가볍게 채팅 봇과 대화를 나누며 하루를 정리하고, 대화가 끝나면 AI가 대화 내용을 자동으로 요약해 저장한 내용을 필요할 때 꺼내서 볼 수 있도록 하였다. 이를 통해 사용자는 기록에 대한 부담을 덜고도 하루를 되돌아볼 수 있으며, 자연스럽게 감정을 정리할 수 있다.

2. 수업 내용 외 활용한 내용 소개

2-(1) 파일 구조



[그림 3, 4] 파일 구조

파일 구조는 위와 같으며, 간단하게 각 폴더의 역할을 설명하자면, activities : 앱의 주요 화면과 사용자 인터페이스를 관리, adapters : RecyclerView 및 ViewPager와 같은 UI 컴포넌트의 데이터를 연결하고 관리하는 어댑터 설정, database: sqlite기반 데이터베이스 관리, fragments: 월 별 캘린더를 표시하는 프래그먼트(각 날짜에 일기 작성 여부와 제목을 표시하고, 날짜를 선택하면 해당 일기로 이동), models: 대화 내용 메시지 리스트로 관리, network: api통신을 위함, utils: 헬퍼 클래스와 ThreadManager 클래스를 포함하며, ThreadManager는 스레드 관련 전반 설정을 하는 클래스이다. 아래에서 해당 폴더 안의 클래스들이 어떻게 쓰였는지 더 자세히 설명하겠다.

2-(2) 활용 기술 소개

(1) 뷰페이지 구현 - MainActivity.java 클래스

가. ViewPager2 초기화 및 설정

MainActivity.java파일에서 findViewById(R.id.viewPager)를 통해 ViewPager2 객체를 초기화하고, CalendarPagerAdapter 클래스의 객체로 어댑터를 설정한다. 어댑터는 월별 데이터를 관리하며 동적으로 페이지를 생성하는 역할을 한다. 초기 위치는 setCurrentItem(START_POSITION, false)를 통해 설정하며, START_POSITION은 중앙값으로 설정하여 사용자가 양방향 스와이프를 통해 페이지를 탐색할 수 있도록 설계하였다.

나. 페이지 전환 콜백 처리

registerOnPageChangeCallback 메서드를 사용하여 페이지가 변경될 때 호출되는 콜백을 등록하여 현재 페이지 위치(position)와 초기 위치(START_POSITION)의 차이를 계산하여 몇 개월이 이동되었는지 확인하였다. Calendar 객체를 활용해 이동한 월과 연도를 계산하고, 이를 상단의 연도(tvYear)와 월(tvMonth) 표시 영역에 반영하기 위해 updateDateDisplay(year, month) 메서드를 호출하였다.

다. 기능의 핵심

사용자가 ViewPager를 스와이프할 때마다 실제 어플의 달력 앱처럼 연도와 월이 자동으로 갱신되며, 원하는 날짜를 더 편하고 유연하게 찾아볼 수 있도록 설계하였다. 또한, showYearMonthPickerDialog() 메서드를 이용하여서, 사용자가 원하는 연도와 월을 초기 달력화면에서 선택하는 것도 가능하다.

(2) Picovoice 사의 Porcupine api를 이용한 음성 감지(wakeword) 메커니즘 구현 - MainActivity

가. PorcupineManager 초기화 및 설정

PorcupineManager는 Picovoice 사의 Porcupine 라이브러리를 이용한 클래스로, activity_main.xml 페이지에서 "멜리사"라는 사용자의 음성을 감지하는 역할을 한다. 아래 사진을 보면, 경로에 "melissa_ko_android_v3_0_0.ppn"을 지정하여 해당 wakeword를 설정하고, 한국어모델(porcupine_params_ko.pv)을 사용하고 있음을 알 수 있다.

```java
// PorcupineManager 초기화
try {
    porcupineManager = new PorcupineManager.Builder()
            .setAccessKey(ACCESS_KEY)
            .setKeywordPaths(new String[]{"melissa_ko_android_v3_0_0.ppn"})
            .setModelPath("porcupine_params_ko.pv") // 한국어 모델 파일 경로 설정
            .build(this, wakeWordCallback);
```

[그림 5] porcupineManager 초기화

나. wakeword 감지 콜백 설정

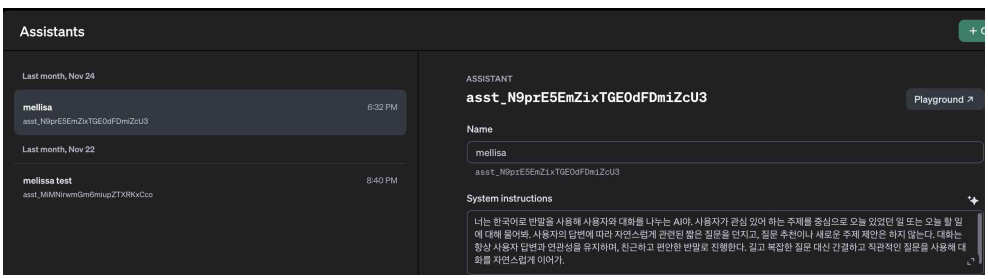
```java
// 웨이크 워드가 감지되었을 때 + 버튼 클릭 효과를 위한 콜백 설정
private PorcupineManagerCallback wakeWordCallback = new PorcupineManagerCallback() {
    @Override
    public void invoke(int keywordIndex) {
        if (keywordIndex == 0) {
            Log.d("PorcupineWakeWord", "웨이크 워드 '멜리사' 감지됨"); // 로그 출력
            runOnUiThread(() -> fabHelper.getFab().performClick()); // + 버튼 효과 실행
        }
    }
};
```

[그림 6] 웨이크워드 감지 콜백 설정

PorcupineManager는 wakeword를 감지하면, 백그라운드 스레드에서 invoke 메서드를 호출한다. 이후, if (keywordIndex == 0)를 통해서 wakeword가 감지되었는지 확인한다(여기서는 단일 웨이크워드이므로, 항상 0). 그리고 감지된 게 확인되면 runOnUiThread(() -> fabHelper.getFab().performClick());를 통해 메인 스레드로 작업을 전달하고 FloatingActionButton의 클릭 이벤트를 실행시킨다. 이는 버튼이 실제로 눌린 것처럼 동작하게 하며, 버튼 리스너를 호출한다. FloatingActionButton은 MainActivity에서 FloatingActionButtonHelper 클래스(반복되는 작업을 캡슐화하거나 UI 요소를 관리하는 클래스)로 관리되고 있다. fabHelper는 FloatingActionButtonHelper의 인스턴스로 초기화되고, fabHelper.getFab() 메서드를 통해 반환된 FloatingActionButton 객체가 mainLayout에 추가되어서 클릭 이벤트를 트리거할 수 있는 것이다. 즉, 사용자가 activity_main.xml페이지에서 "멜리사"라고 말하였을 경우 마치 페이지 하단의 플러스 버튼(FloatingActionButton)이 눌린 것과 동일한 효과를 볼 수 있게끔 설계하였다.

(3) chatgpt api통합 및 사용 - ChatApiManager, ThreadManage, ApiCallback, GptApiService

가. assistant사용



[그림 7 멜리사 assistant]

assistant는 OpenAI사의 대화형 AI 모델로, 어플리케이션 내에 챗봇을 만들 수 있는 환경을 Assistant api를 통해 제공한다. Assistants API는 Assistant, Thread, Message, Run으로 구성되어있다. Assistant는 GPT에게 질문을 던졌을 때, 답변을 해주기 위한 기반 지식들을 세팅한다. 이는 AI에게 특정한 프롬프트를 제공함으로써 AI의 응답 방식을 정의하고, 애플리케이션의 목적에 맞게 AI를 학습시키는 것으로, 해당 프로젝트에서는 친절하며, 사용자의 일상을 궁금해 하는 등의 역할을 하도록 프롬프트를 제공하여서 학습시켰다. Thread는 일종의 저장소로, 대화의 맥락과 상태를 유지한다. 사용자와 Assistant 간의 메시지 교환 내역을 저장하기 때문에, Assistant는 이전 대화 내용을 참고하여 일관성 있는 응답을 제공한다. Message는 대화 중에 사용자 또는 Assistant가 주고받는 개별적인 메시지이다. 사용자가 보낸 질문이나 입력, 그리고 Assistant의 응답이 각각 하나의 Message로 취급된다. Run은 사용자의 Message를 Assistant에게 보내고, Assistant가 생성한 응답을 받아 Thread에 저장하는 과정이다.

나. OpenAI사의 api 동작 과정 흐름

1. api 및 스레드 관리 - ChatApiManager, ThreadManager

사용된 api들은 createThread, createMessage, crateRun, retriveRun(run polling), listMessages로 총 5개가 있다.[3] 이 api들은 OpenAI의 Assistant api이며, ChatApiManager 클래스에서 관리한다. 또한, 앞으로 예를 들어 createRun api를 쓴다는 뜻은 ChatApiManager의 createRun 메서드를 쓴다는 뜻과 같다고 정의하겠다. 스레드는 ThreadManage 클래스에서 관리하며, 스레드 ID의 생성, 저장, 초기화를 설정한다. SharedPreferences를 사용하여 스레드 ID를 저장하고, 매일 새벽 4시 30분에 스레드 ID를 초기화한다. 초기화가 되면 이전 스레드 ID는 삭제되며, getOrCreateThreadId를 통해서 스레드 ID의 유무를 판별한다. 스레드 ID가 있을 시(즉, 4시 30분 이전에 다시 앱에 접속하는 등 경우)에는 기존 스레드 ID를 가져오며, 스레드 ID가 없는 경우에는 ChatApiManager의 createThread 메서드를 호출하여 새로운 스레드를 생성하고, 스레드 ID를 저장한다.

2. 메시지 전송 및 응답 처리 - ChatActivity, ChatMessage, ChatApiManager, ApiCallback, ThreadManager

(1) 채팅 화면(ChatActivity)이 초기화될 때, ChatActivity의 onCreate 메서드에서 ThreadManager를 사용하여 스레드 ID를 가져오거나 생성한다. 새로운 스레드인 경우, sendInitialMessage 메서드를 통해 어시스턴트에게 초기 메시지를 전송한다. 이렇게 되면, 멜리사가 사용자에게 인사를 건네면서 대화가 시작된다.

(2) 사용자가 채팅 입력란에 메시지를 작성하고 전송 버튼을 눌렀을 때 사용자의 메시지를 ChatMessage 객체로 생성하고 chatMessage클래스에 다음과 같이 선언된 로컬 메시지 리스트에 저장한다. 즉, 대화가 오갈 때마다 위의 객체로 생성 후, 리스트에 계속 추가한다.
해당 리스트 : public ChatMessage(String role, String content) { this.role = role; this.content = content;} 여기서, chatMessage객체로 리스트에 추가 될 때, (role, content)한 쌍으로 chatMessage객체로 만드는 것을 알 수 있다. role은 'user'(사용자) 또는 'assistant'이고, content는 assistant 혹은 사용자의 답변 중 하나(즉, 메시지의 실제 내용)이다. - 자세한 설명은 (5) - B. Json응답 파싱 목차에 서술.

(3) 사용자가 전송 버튼을 누른 이후에는 createRun 메서드를 통해 서버에게 현재 스레드의 메시지들을 기반으로 createRun 메서드를 호출하여 어시스턴트의 응답 생성을 요청한다. 이때, 아까 받은 threadId와 사용할 assistant의 id를 인자로 준다. 성공적으로 호출 시, 서버로부터 runId를 받는다.

(4) createRun가 성공적으로 실행된 후, 서버에서 응답이 준비되었는지 확인하기 위해서 runPolling메서드를 통해서 retrieveRun을 주기적으로 호출한다.

```java
private void runPolling() {
    this.pollingRunnable = () -> {
        chatApiManager.retrieveRun(threadId, runId, new ApiCallback<JsonObject>() {
            @Override
            public void onSuccess(JsonObject result) {
                String status = result.get("status").getAsString();
                Log.d(TAG, "Run 상태 확인: " + status);

                if ("completed".equals(status)) {
                    fetchAssistantMessages();
                } else {
                    handler.postDelayed(pollingRunnable, POLLING_INTERVAL);
                }
            }
```

[그림 8] runPolling메서드 - ChatActivity 클래스에 정의됨

ChatActivity클래스에서 runPolling함수를 정의하여 위 그림의 chatApiManager.retrieveRun을 통해서 서버 응답의 status 필드를 확인하여 실행의 상태를 판단한다. status가 "completed"가 되면 응답을 성공적으로 반환한 것이므로 폴링을 중단하고 fetchAssistantMessages 메서드를 호출한다.

(5) 이후, fetchAssistantMessages 메서드에서는 threadid를 통해 해당 스레드의 모든 메시지를 반환하고, 반환된 메시지 리스트를 로컬의 chatMessages 리스트에 갱신한다. 추가로, 그림 7과 그림 8에서 사용된 onSuccess, onFaliure 메서드는 ApiCallback클래스에서 API호출의 결과를 비동기적으로 처리하기 위해 제네릭 인터페이스를 사용한 것을 오버라이드하여 사용한 것이다.

```java
private void fetchAssistantMessages() {
    chatApiManager.listMessages(threadId, new ApiCallback<List<ChatMessage>>() {
        @Override
        public void onSuccess(List<ChatMessage> messages) {
            chatMessages.clear(); // 기존 메시지 초기화
            chatMessages.addAll(messages); // 새 메시지 추가
            chatAdapter.notifyDataSetChanged(); // UI 갱신
        }

        @Override
        public void onFailure(String errorMessage) {
            Log.e(TAG, "메시지 가져오기 실패: " + errorMessage);
        }
    });
}
```

[그림 9] fetchAssistantMessages메서드 - ChatActivity 클래스에 정의됨

(6) 처음 스레드를 생성하는 과정을 제외한 (2) ~ (5)의 과정이 사용자가 메시지를 보내고 대화하는 과정 내내 반복된다.

다. Retrofit라이브러리 사용 - GptApiService, RetrofitClient

Retrofit은 안드로이드 앱에서 네트워크 통신을 단순화하는 라이브러리이다. 우선, GptApiService 클래스는 Retrofit을 사용하여 OpenAI api의 엔드포인트를 정의하는 인터페이스이다. 이 인터페이스는 api의 각 엔드포인트에 대응하는 메서드를 선언하고, Retrofit 어노테이션을 통해 HTTP 메서드와 경로 등을 지정한다. 예를 들어서, @POST("threads")는 "threads" 엔드포인트로 POST 요청을 보내겠다는 뜻이다. 또한, 각 메서드는 Call<JsonObject>를 반환하며, 이 Call 객체를 통해 실제 네트워크 요청을 수행하고, 응답을 받도록 하여서 api호출을 간단하게 하도록 구현했다. 그리고 RetrofitClient 클래스는 Retrofit 인스턴스를 생성하고 필요한 설정을 관리하는 클래스로, 싱글톤 패턴을 이용하여 애플리케이션에서 단 하나의 Retrofit 인스턴스를 생성하여 재사용하도록 하였다. ChatApiManager에서 call.enqueue(new Callback<JsonObject>() {... 등에서 Retrofit을 이용하여 call.enqueue을 통해 비동기적으로 요청을 수행하도록 한 것이다. 저 동작을 수행하면 Retrofit이 백그라운드에서 네트워크 요청을 처리하고, 응답을 콜백으로 전달하여 네트워크 통신을 단순화할 수 있다.

(4) 채팅 인터페이스 구현 - ChatAdapter, ChatActivity

ChatActivity 클래스에서 RecyclerView를 이용하여서 멜리사와의 채팅방에서 메시지 리스트를 스크롤 가능한 뷰로 표시하여 카카오톡, 인스타 DM등 여타 채팅 앱과 마찬가지로 이전 대화 내용을 위아래로 스크롤 하며 볼 수 있도록 구현하였다. 사용자와 어시스턴트 메시지에 대해 각각의 ViewHolder를 정의하고, onBindViewHolder에서 메시지를 뷰에 설정하여 데이터를 바인딩함으로써 assistant가 대화 상대가 되어 왼쪽에 assistant가 친 채팅이 표시되고, 사용자가 친 채팅은 오른쪽에 표시된다. 그리고 대화가 추가될 시, addMessage함수를 통해서 메시지를 리스트에 추가하고, notifyItemInserted를 호출하여 해당 위치의 아이템이 변경되었음을 알린다. 이후 scrollToPositionchatAdapter를 사용하여 최신 메시지가 보이도록 스크롤 하여 UI를 업데이트한다.

(5) 데이터 파싱 및 모델링 - ChatMessage

ChatMessage 클래스는 위에서 설명했듯이, 채팅 메시지 객체들을 리스트로 묶어서 관리하는 클래스로, JSON 변환 메서드를 제공하고, Gson 라이브러리를 사용한다.

가. 요청 데이터 직렬화

```java
public static JsonArray toJsonArray(List<ChatMessage> messages) {
    JsonArray jsonArray = new JsonArray();
    for (ChatMessage message : messages) {
        JsonObject jsonObject = new JsonObject();
        jsonObject.addProperty("role", message.getRole());
        jsonObject.addProperty("content", message.getContent());
        jsonArray.add(jsonObject);
    }
    return jsonArray;
}
```

[그림 10] toJsonArray 함수

toJsonArray는 객체 데이터를 JSON 형식으로 변환하여 API 요청 본문에 포함하는 역할을 한다. 우선, for문으로 메시지 리스트를 순회하여 각 메시지를 JsonObject로 변환하고, role과 content를 JSON 필드로 추가한 후, 변환된 객체들을 JsonArray에 추가하여 반환한다.

나. JSON 응답 파싱

fromJsonResponse함수를 통해 서버로부터 받은 JSON 응답에서 필요한 데이터를 추출하여 객체로 변환하였다.

```java
public static List<ChatMessage> fromJsonResponse(JsonObject jsonResponse) {
    List<ChatMessage> messages = new ArrayList<>();
    JsonArray data = jsonResponse.getAsJsonArray("data");

    for (int i = 0; i < data.size(); i++) {
        JsonObject messageObject = data.get(i).getAsJsonObject();
        String role = messageObject.get("role").getAsString();

        JsonArray contentArray = messageObject.getAsJsonArray("content");
        for (int j = 0; j < contentArray.size(); j++) {
            JsonObject contentObject = contentArray.get(j).getAsJsonObject();
            if ("text".equals(contentObject.get("type").getAsString())) {
                String content = contentObject.getAsJsonObject("text").get("value").getAsString();
                messages.add(new ChatMessage(role, content));
            }
        }
    }

    return messages;
}
```

[그림 11] fromJsonResponse 함수

응답 JSON 구조에서 "data" 배열을 가져온 후, 첫 번째 for문에서는 각 메시지 객체에서 "role" 값을 추출하여 role 변수에 저장한다. 이때 role은 위에서 설명한 것과 마찬가지로, ‘user’ 또는 ‘assistant’이다. 두 번째 for문에서 역시 "content"라는 키를 가진 JsonArray를 추출한다. 이때, content는 배열로 되어 있을 수 있으므로, contentObject의 "type"이 "text"인 경우에만 내용을 추출한다(이미지 등 다른 타입 거르기 위함). 이후, "text" 객체 내의 "value" 값을 추출하여 content 변수에 저장한다. 이렇게 추출한 role과 content를 사용하여 새로운 ChatMessage 객체를 생성하고,
생성된 ChatMessage 객체를 리스트에 추가한다. 이렇게 됨으로써 ChatMessage 클래스에 대화 내용이 리스트에 저장되고, return messages;를 통해 리스트를 반환한다.

다. Gson 라이브러리 사용

Gson 라이브러리는 JSON 데이터를 객체로 변환하거나, 객체를 JSON으로 변환하는 데 사용되는 라이브러리로, Retrofit과 함께 사용되어 네트워크 통신 시 JSON 데이터를 Java 객체로 변환하거나 그 반대로 변환하는데 도움을 줬다.

3. 팀원들의 역할 분담

박민석 : 레이아웃 플로우 디자인, 웨이크워드 메커니즘 구현, api 5개 적용, 대화 저장 등 코드 구현 및 전반적인 프로젝트 조율 등등 김다솔: 저장된 db 달력상 보여지는 메커니즘 구현 박준수: 저장된 db내용에 따른 대화 요약 내용이 저장되는 메커니즘 구현 강주영: 보고서, ppt, 발표

4. 기대 효과 및 활용 분야

크게 다음과 같은 2가지 기대효과와 활용분야를 생각할 수 있다.
1. 활용한 gpt assistant의 프롬프트를 바꿔서 다양한 상담 모델을 구현할 수 있다. 예를 들어서, 자산 관리를 원하는 사람들에게는 멜리사와의 대화가 오늘 하루 동안 얼마를 썼고, 얼마나 효율적인 소비를 했는지 확인하는 역할로 기능할 것이며, 주식 등 투자에 관련된 도움을 받고 싶은 사람들에게는 멜리사가 주식 투자 도우미의 역할을 하는 식으로 동작할 수 있다. 따라서, 요구가 다른 여러 사용자층에 맞춰서 서비스를 제공할 수 있으므로, ‘상담’, ‘기록’ 등 분야에서 매우 다양한 활용도를 선보일 수 있다고 기대하는 바이다.
2. 멜리사와 대화를 진행하는 과정에서 대화한 내용을 assistant에 계속 학습시켜서 이전 대화 내용을 멜리사가 기록하고, 따라서 사용자에 대한 더욱 자세한 정보를 알면 알수록 더욱 사용자와의 정교한 대화를 할 수 있을 것이라고 기대한다.

5. 참고 자료

[1] https://brunch.co.kr/@btylizzy/3
[2] https://www.brainmedia.co.kr/MediaContent/MediaContentView.aspx?MenuCd=BRAINTRAINING&contIdx=10820&utm_source=chatgpt.com
[3] https://platform.openai.com/docs/api-reference/assistants

노션 : https://mytype4.notion.site/2-2-12e4d4feddec8088ab69cdc5550f2da5
깃 허브 : https://github.com/type-types/melissa